Sentiment Analysis of Product Reviews

Presented by:
Sebin George Lubaba Sadiya NP Manasa Muraleedharan.E

Introduction

- Performs sentiment analysis on product reviews
- Classifies reviews as Positive, Negative, or Neutral
- Uses Natural Language Processing (NLP) techniques
- Applies Machine Learning and Deep Learning models
- Compares SVM and DistilBERT performance
- Provides a user-friendly interface using Streamlit

Dataset Overview

- Dataset includes: text, label, stars, sentiment
- Total samples: 10,000 reviews
- No missing values in any column
- Data types: integer, float, and text
- Labels (0–4) converted to star ratings (1–5)
- Mean rating around 3 (balanced distribution)
- Sentiment mapping:
  - 1–2: Negative
  - 3: Neutral
  - 4–5: Positive
- Data is clean and suitable for model training

Data Preparation

- Loaded Yelp Review dataset using datasets library
- Extracted text and label features
- Reduced dataset to 10,000 samples for faster processing
- Converted numerical labels to star ratings
- Defined sentiment categories based on ratings

SVM + TF-IDF

- Applied TF-IDF for feature extraction
- Converted text into numerical vectors
- Used SVM for classification
- Predicted Positive, Negative, Neutral

- Accuracy: 75%
- Strong performance for Positive and Negative classes
- Neutral class shows lower performance

SVM + TF-IDF Results

SVM + TF-IDF Performance

	precision	recall	f1-score	support
Negative	0.78	0.86	0.82	805
Neutral	0.53	0.37	0.44	396
Positive	0.80	0.84	0.82	799
accuracy	**0.75**	**0.75**	**0.75**	**2000**
macro avg	0.70	0.69	0.69	2000
weighted avg	0.70	0.69	0.64	2000
positive	0.74	0.82	0.44	2000

Model Comparison

- Compared SVM (TF-IDF) with DistilBERT
- SVM works well for simple text data
- DistilBERT captures contextual meaning
- SVM is faster and lightweight
- DistilBERT gives higher accuracy
- Helps select the best model

What is Streamlit?

A Python framework for building interactive web apps — no HTML or JavaScript needed.

- **Instant hot-reload**
  Changes reflect without restart
- **Pure Python**
  No front-end knowledge required
- **Built-in widgets**
  Sliders, text areas, radio buttons
- **Free cloud hosting**
  Deploy via Streamlit Community Cloud

```
st.title("Sentiment Analyzer")
model_choice = st.radio("Model", ["DistilBERT", "SVM"])
review_text = st.text_area("Review", placeholder =
```

```
if st.button("Analyse sentiment"):  sentiment,
```

App Features & Interface

Model Selector

Toggle between DistilBERT and SVM via radio buttons at runtime

Text Input

160 px text area for pasting product reviews

Confidence Bar

Animated progress bar showing prediction confidence percentage

Dark UI Theme

Custom CSS — `#0f0f13` background, DM Serif Display font

Token Warning

Alerts user when input exceeds 128 tokens (DistilBERT limit)

3-Class Output

Positive / Negative / Neutral with distinct color coding

Deployment Steps — Streamlit Cloud

1. **Prepare Repository**
   Push `app.py`, `requirements.txt`, and model files (`tfidf_vectorizer.pkl`, `svm_model.pkl`, `best_distilbert.pt`) to GitHub
2. **requirements.txt**
   Include: `streamlit`, `scikit-learn`, `torch`, `transformers` with pinned versions
3. **Connect GitHub to Streamlit Cloud**
   Sign in at `share.streamlit.io` → New App → Select repo, branch, and `app.py` as entry point
4. **Deploy & Monitor**
   Click Deploy — Streamlit installs packages and launches the app; view real-time logs for errors
5. **Share the App URL**
   Get a public `*.streamlit.app` URL — share instantly, no server

Live App — Demo & Key Takeaways

Key Takeaways

- ✓ Fully interactive web app in pure Python — no JavaScript
- Dual-model inference: SVM and DistilBERT selectable at runtime
- Custom dark-theme CSS with confidence bar & token warnings
- Deployed on Streamlit Community Cloud — free public URL
- `@st.cache_resource` used for fast repeated model inference